Mithril in Game-based Syntax

Wuyunsiqin[1], Bingsheng Zhang[1], Markulf Kohlweiss[2]
[1]Zhejiang University, {oyun, bingsheng}@zju.edu.cn
[2]University of Edinburgh & Input Output, mkohlwei@ed.ac.uk

January 20, 2026

1 Preliminaries

1.1 Notations

There are some basic notations and conventions used throughout this work. The security parameter is denoted by $\lambda \in \mathbb{N}$. For any finite set S, we write $|S|$ for its cardinality. We use the notation $x \xleftarrow{\$} S$ to indicate that x is sampled uniformly at random from the finite set S. A function $\mathsf{negl} : \mathbb{N} \to \mathbb{R}_{\geq 0}$ is called *negligible* if for every polynomial $p(\cdot)$ there exists $N \in \mathbb{N}$ such that for all $\lambda \geq N$ it holds that $\mathsf{negl}(\lambda) < 1/p(\lambda)$. We write $\mathsf{negl}(\lambda)$ to denote an unspecified negligible function in the security parameter λ.

1.2 BLS Signature Scheme

Let λ be the security parameter. We assume a bilinear group generation algorithm which, on input 1^λ, outputs $(q, G_1, G_2, G_T, g_1, g_2, e)$, where G_1, G_2, G_T are cyclic groups of prime order q, $g_1 \in G_1$, $g_2 \in G_2$ are generators, and $e : G_1 \times G_2 \to G_T$ is a bilinear pairing. Let $H : \{0,1\}^* \to G_1$ be a secure hash function modeled as a random oracle. We write the BLS signature scheme as $\Pi_{\mathsf{BLS}} := (\mathsf{Setup}, \mathsf{KGen}, \mathsf{Sign}, \mathsf{Ver})$.

- $\mathsf{pp}_{\mathsf{BLS}} \leftarrow \mathsf{Setup}(1^\lambda)$: Run the group generator to obtain $(q, G_1, G_2, G_T, g_1, g_2, e)$ and pick a suitable H. Output $\mathsf{pp}_{\mathsf{BLS}} := (q, G_1, G_2, G_T, g_1, g_2, e, H)$.
- $(sk, vk) \leftarrow \mathsf{KGen}(\mathsf{pp}_{\mathsf{BLS}})$: Sample a secret key $sk \xleftarrow{\$} \mathbb{Z}_q$, and set $vk = g_2^{sk} \in G_2$. Output the key pair (sk, vk).
- $\sigma \leftarrow \mathsf{Sign}(\mathsf{pp}_{\mathsf{BLS}}, sk, msg)$: On input $\mathsf{pp}_{\mathsf{BLS}}$, the secret key sk, and the message $msg \in \{0,1\}^*$, compute $h \leftarrow H(msg) \in G_1$ and set $\sigma = h^{sk} \in G_1$. Output the signature σ.
- $b \leftarrow \mathsf{Ver}(\mathsf{pp}_{\mathsf{BLS}}, vk, msg, \sigma)$, where $b \in \{0,1\}$: Compute $h \leftarrow H(msg)$ and output $b = 1$ iff $e(\sigma, g_2) = e(h, vk)$, and $b = 0$ otherwise.

1.3 Merkle Tree

Let $H_{\mathsf{MT}} : \{0,1\}^* \to \{0,1\}^\lambda$ be a collision-resistant hash function. We denote the Merkle tree primitive by $\Pi_{\mathsf{MT}} := (\mathsf{Create}, \mathsf{Check})$.

- $\mathsf{rt} \leftarrow \mathsf{Create}(\mathbf{m})$: On input a vector of messages $\mathbf{m} = (m_1, \ldots, m_N)$, set leaves $\ell_i := H_{\mathsf{MT}}(m_i)$ and output the Merkle root rt.
- $b \leftarrow \mathsf{Check}(\mathsf{rt}, m, \mathsf{p})$, where $b \in \{0,1\}$: On input a root rt, a message m, and an authentication path p, re-compute the root from the leaf $\ell := H_{\mathsf{MT}}(\mathsf{m})$ and p. Output $b = 1$ iff the re-computed root equals to rt, and $b = 0$ otherwise.

1.4 SNARK

Let $x \in \mathcal{L}$ be a statement, $w \in \mathcal{W}$ be a witness and $(x, w) \in \mathcal{R}$ be an NP relation.We denote a succinct non-interactive argument (SNARK) as a tuple $\Pi_{\mathsf{SNARK}} := (\mathsf{Setup}, \mathsf{Prove}, \mathsf{Verify})$.

- $\mathsf{pp}_{\mathsf{SNARK}} \leftarrow \mathsf{Setup}(1^\lambda, \mathcal{R})$: outputs public parameters $\mathsf{pp}_{\mathsf{SNARK}}$ for relation $\mathcal{R}$.
- $\pi \leftarrow \mathsf{Prove}(\mathsf{pp}_{\mathsf{SNARK}}, x, w)$: outputs a proof π for statement x with witness w.
- $b \leftarrow \mathsf{Verify}(\mathsf{pp}_{\mathsf{SNARK}}, x, \pi)$, where $b \in \{0,1\}$: outputs 1 if π is accepted for x, and 0 otherwise.

We require that Π_{SNARK} satisfies completeness, knowledge soundness, and succinctness.

Completeness. For all $(x, w) \in \mathcal{R}$ and all $\lambda \in \mathbb{N}$, if $\mathsf{pp}_{\mathsf{SNARK}} \leftarrow \mathsf{Setup}(1^\lambda, \mathcal{R})$ and $\pi \leftarrow \mathsf{Prove}(\mathsf{pp}_{\mathsf{SNARK}}, x, w)$, then $\Pr[\mathsf{Verify}(\mathsf{pp}_{\mathsf{SNARK}}, x, \pi) = 1] = 1$.

Knowledge soundness. For every PPT adversary $\mathcal{A}$, there exists a PPT extractor $\mathcal{X}$ such that for any $(\mathsf{pp}_{\mathsf{SNARK}}, x)$ output by $\mathsf{Setup}(1^\lambda, \mathcal{R})$ and any proof π output by $\mathcal{A}(\mathsf{pp}_{\mathsf{SNARK}})$, we have $\Pr[\mathsf{Verify}(\mathsf{pp}_{\mathsf{SNARK}}, x, \pi) = 1 \wedge (x, w) \notin \mathcal{R}$ where $w \leftarrow \mathcal{X}^{\mathcal{A}}(\mathsf{pp}_{\mathsf{SNARK}}, x, \pi)] \leq \mathsf{negl}(\lambda)$. Equivalently, whenever $\mathcal{A}$ outputs an accepting proof for x, the extractor can output a witness w such that $(x, w) \in \mathcal{R}$ except with negligible probability.

Remark. In this work, we need black-box extractability. Many SNARK constructions in the literature have a black-box extractor.

Succinctness. There exists a polynomial $\mathsf{poly}(\cdot)$ such that for all λ and all $(x, w) \in \mathcal{R}$, the proof size $|\pi|$ and verification time of $\mathsf{Verify}(\mathsf{pp}_{\mathsf{SNARK}}, x, \pi)$ are bounded by $\mathsf{poly}(\lambda, \log |x|)$.

2 Stake-based threshold multi-signature

2.1 Syntax

We consider a stake-based threshold multi-signature (STMS) scheme whose acceptance is determined on a per-stake basis via a "t wins in n lotteries" parametrization, for some integers $t, n \in \mathbb{N}$.

$$\Pi^{t,n}_{\mathsf{STMS}} = (\mathsf{Setup}, \mathsf{KGen}, \mathsf{AKey}, \mathsf{ACheck}, \mathsf{Sig}, \mathsf{Ver}, \mathsf{ASig}, \mathsf{AVer}).$$

- $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$: on input a security parameter 1^λ, outputs public system parameters pp. These parameters are implicit inputs to all other algorithms.
- $(\mathsf{sk}, \mathsf{vk}) \leftarrow \mathsf{KGen}(\mathsf{pp})$: on input the public parameters pp, generates a signing/verification key pair $(\mathsf{sk}, \mathsf{vk})$ for a single party.
- $\mathsf{avk} \leftarrow \mathsf{AKey}(\mathbf{vk}, \mathbf{w})$: given a list of verification keys $\mathbf{vk} := (\mathsf{vk}_1, \ldots, \mathsf{vk}_N)$ and a matching stake vector $\mathbf{w} := (w_1, \ldots, w_N)$, where $N \in \mathbb{N}$ denotes the number of users, deterministically produces an aggregate verification key avk.
- $b \leftarrow \mathsf{ACheck}(\mathsf{avk}, \mathbf{vk}, \mathbf{w})$, where $b \in \{0, 1\}$: deterministically checks whether avk is a valid aggregate verification key for $(\mathbf{vk}, \mathbf{w})$. Output $b = 1$ if it accepts and $b = 0$ otherwise.
- $\sigma \leftarrow \mathsf{Sig}(\mathbf{vk}, \mathbf{w}, \mathsf{sk}, j, m)$: attempts to produce a signature σ for message m and lottery index $j \in [n]$ under verification key vk, secret key sk, and stake w. It outputs $\perp$ if the user is ineligible for (m, j) under stake w.
- $b \leftarrow \mathsf{Ver}(\mathbf{vk}, \mathbf{w}, i, j, m, \sigma)$, where $b \in \{0, 1\}$: given a verification key vk, a stake value w, a lottery index j, a message m, and a signature σ, deterministically checks whether σ is a valid signature for (m, j) under (vk_i, w_i) and outputs $b = 1$ if it accepts and $b = 0$ otherwise.
- $\tau \leftarrow \mathsf{ASig}(\mathbf{vk}, \mathbf{w}, m, \{(\mathsf{vk}_i, \sigma_i)\}_{i=1}^t)$: given $(\mathbf{vk}, \mathbf{w})$, a message m, and a multiset $\{(vk_i, \sigma_i)\}_{i=1}^t$ of signatures, attempts to produce an aggregate signature τ; it outputs $\perp$ if the provided inputs are insufficient or inconsistent.
- $b \leftarrow \mathsf{AVer}(\mathsf{avk}, m, \tau, t)$, where $b \in \{0, 1\}$: given a message m, an aggregate verification key avk, an aggregate signature τ, and a stake threshold parameter T, outputs $b = 1$ if τ is accepted as a valid aggregate (stake-based threshold) multi-signature on m under avk and threshold T, and $b = 0$ otherwise.

2.1.1 Correctness

Let $\Pi_{\mathsf{STMS}} = (\mathsf{Setup}, \mathsf{KGen}, \mathsf{AKey}, \mathsf{ACheck}, \mathsf{Sig}, \mathsf{Ver}, \mathsf{ASig}, \mathsf{AVer})$ be a stake-based threshold multi-signature (STMS) scheme as specified above. Let $\mathcal{M}$ denote the message space.

Definition (Correctness). We say that Π_{STMS} is *correct* if for every security parameter $\lambda \in \mathbb{N}$, every number of users $N \in \mathbb{N}$, every vector of verification keys $\mathbf{vk} := (\mathsf{vk}_1, \ldots, \mathsf{vk}_N)$ generated by KGen, every stake vector $\mathbf{w} := (w_1, \ldots, w_N) \in (\mathbb{Z}^+)^N$, every message $m \in \mathcal{M}$, every stake threshold parameter $T \in \mathbb{N}$, and every lottery index $j \in \{1, ..., n\}$, where n is lottery index range. The following properties hold except with negligible probability in λ over the randomness of all algorithms involved.

- **Correctness of aggregate key.** For every vector of verification keys $\mathbf{vk} := (\mathsf{vk}_1, \ldots, \mathsf{vk}_N)$ generated by KGen, and every stake vector $\mathbf{w} := (w_1, \ldots, w_N) \in (\mathbb{Z}^+)^N$, let $\mathsf{avk} \leftarrow \mathsf{AKey}(\mathbf{vk}, \mathbf{w})$, we have

$$\Pr[\mathsf{ACheck}(\mathsf{avk}, \mathbf{vk}, \mathbf{w}) = 1] = 1 - \mathsf{negl}(\lambda).$$

- **Correctness of individual signatures.** For every party index $i \in [N]$, let $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$, $(\mathsf{sk}_i, \mathsf{vk}_i) \leftarrow \mathsf{KGen}(\mathsf{pp})$, $\sigma_i \leftarrow \mathsf{Sig}(\mathbf{vk}, \mathbf{w}, \mathsf{sk}_i, j, m)$. If $\sigma_i \neq \perp$, we have

$$\Pr[\mathsf{Ver}(\mathbf{vk},\ \mathbf{w}, i, j, m, \sigma_i) = 1] = 1 - \mathsf{negl}(\lambda).$$

- **Correctness of aggregate signatures.** Let $t \in \mathbb{N}$, and let $\{I_\eta\}_{\eta \in [t]}$ be a set of indices, let $(\mathsf{sk}_{I_\eta}, \mathsf{vk}_{I_\eta})$ be generated by $(\mathsf{sk}_{I_\eta}, \mathsf{vk}_{I_\eta}) \leftarrow \mathsf{KGen}(\mathsf{pp})$ and let w_{I_η} be the corresponding stake value. For a message m and a lottery index j, let $\sigma_{I_\eta} \leftarrow \mathsf{Sig}(\mathsf{vk}_{I_\eta}, w_{I_\eta}, \mathsf{sk}_{I_\eta}, j, m)$ be such that $\sigma_{I_\eta} \neq \perp$. Let

$$\tau \leftarrow \mathsf{ASig}(\mathbf{vk}, \mathbf{w}, m, \{(\mathsf{vk}_{I_\eta}, \sigma_{I_\eta})\}_{\eta=1}^{t}) \quad \text{and} \quad \mathsf{avk} \leftarrow \mathsf{AKey}(\mathbf{vk}, \mathbf{w}).$$

  we have

$$\Pr[\mathsf{AVer}(\mathsf{avk}, m, \tau, T) = 1] = 1 - \mathsf{negl}(\lambda).$$

2.1.2 Security

Stake-based unforgeability α-UF-CMA.
Let $\Pi_{\mathsf{STMS}} = (\mathsf{Setup}, \mathsf{KGen}, \mathsf{AKey}, \mathsf{ACheck}, \mathsf{Sig}, \mathsf{Ver}, \mathsf{ASig}, \mathsf{AVer})$ be an STMS scheme as above, and consider the following experiment for an adversary A:

α-UF-CMA experiment

1. Runs $pp \leftarrow \mathsf{Setup}(1^\lambda)$. Let $\mathcal{A}$ takes input as pp, and outputs a stake vector $\mathbf{w} := (w_1, \ldots, w_N) \in (\mathbb{Z}^+)^N$. For each $i \in [N]$ samples a key pair $(\mathsf{sk}_i, \mathsf{vk}_i) \leftarrow \mathsf{KGen}(\mathsf{pp})$.

   Fix the number of total lottery indices n, and the threshold t. Computes the total stake $W := \sum_{i=1}^{N} w_i$ and then gives $(\mathbf{vk}, n, t)$ to $\mathcal{A}$, where $\mathbf{vk} := (\mathsf{vk}_1, \ldots, \mathsf{vk}_N)$. It then initializes an empty set $\mathcal{C} \subseteq [N]$ and an empty set $\mathcal{Q}_{\mathsf{topic}}$ to record signing queries.

2. The adversary $\mathcal{A}$ may make corruptions, key substitution and signing oracle queries at any time before it outputs.

   - Corruption. The adversary $\mathcal{A}$ queries an index $i \in [N]$. If $i \notin \mathcal{C}$, $\mathcal{C} = \mathcal{C} \cup \{i\}$ add i to $\mathcal{C}$ and returns sk_i to $\mathcal{A}$.
   - Key substitution. The adversary $\mathcal{A}$ modifies a verification key $\mathbf{vk}[i] \leftarrow \mathsf{vk}_i^*$. If $i \notin \mathcal{C}$, $\mathcal{C} = \mathcal{C} \cup \{i\}$.
   - Signing oracle queries. The adversary $\mathcal{A}$ queries a user index $i \in [N]$, and a message $m \in \mathcal{M}$. If $i \notin \mathcal{C}$, Computes a set of $\sigma \leftarrow \mathsf{Sig}(\mathbf{vk}, \mathbf{w}, \mathsf{sk}_i, j, m)$ for every lottery index $j \in [n]$ and returns the set to $\mathcal{A}$. Records $(i, m.\mathsf{topic})$ in the set $\mathcal{Q}_{\mathsf{topic}}$. (Here each message $m \in \mathcal{M}$ is assumed to be a structured pair $m = (\mathsf{topic}, \mathsf{body})$.)

3. The adversary $\mathcal{A}$ outputs $(\mathsf{avk}^*, \mathbf{vk}^*, m^*, \tau^*)$, where $\mathbf{vk}^* = (\mathsf{vk}_1^*, \ldots, \mathsf{vk}_N^*)$ is a vector of verification keys, $m^* \in \mathcal{M}$ is a message and τ^* is an aggregate signature. Let $\mathcal{C}^* := \{i \mid i \in \mathcal{C} \text{ or } \exists\, i \text{ s.t. } (i, m^*.\mathsf{topic}) \in \mathcal{Q}_{\mathsf{topic}}\}$.

   Return 1 if $\mathsf{ACheck}(\mathsf{avk}^*, \mathbf{vk}^*, \mathbf{w}) = 1, \mathsf{AVer}(\mathsf{avk}^*, m^*, \tau^*, t) = 1$, and $\sum_{i \in C^*} w_i < \alpha \sum_{i \in [N]} w_i$.

We say that Π_{STMS} is α-UF-CMA secure if for every PPT adversary $\mathcal{A}$:

$$\mathsf{Adv}_{\mathcal{A},\mathsf{STMS}}^{\alpha\text{-}\mathbf{UF\text{-}CMA}}(\lambda) := \Pr[\alpha\text{-}\mathbf{UF\text{-}CMA}(\lambda) = 1] < \mathsf{negl}(\lambda).$$

Static stake-based unforgeability α-UF-CMA-s.
Let $\Pi_{\mathsf{STMS}} = (\mathsf{Setup}, \mathsf{KGen}, \mathsf{AKey}, \mathsf{ACheck}, \mathsf{Sig}, \mathsf{Ver}, \mathsf{ASig}, \mathsf{AVer})$ be an STMS scheme as above, and consider the following experiment for an adversary A:

α-UF-CMA-s experiment

1. Runs $pp \leftarrow \mathsf{Setup}(1^\lambda)$. Let $\mathcal{A}$ takes input as pp, and outputs a stake vector $\mathbf{w} := (w_1, \ldots, w_N) \in (\mathbb{Z}^+)^N$. For each $i \in [N]$ samples a key pair $(\mathsf{sk}_i, \mathsf{vk}_i) \leftarrow \mathsf{KGen}(\mathsf{pp})$.

   Fix the number of total lottery indices n, and the threshold t. Computes the total stake $W := \sum_{i=1}^{N} w_i$ and then gives $(\mathbf{vk}, n, t)$ to $\mathcal{A}$, where $\mathbf{vk} := (\mathsf{vk}_1, \ldots, \mathsf{vk}_N)$. It then initializes an empty set $\mathcal{C} \subseteq [N]$ and an empty set $\mathcal{Q}_{\mathsf{topic}}$ to record signing queries.

2. The adversary $\mathcal{A}$ may make corruptions and key substitutions only before its first signing oracle query.

   - Corruption. The adversary $\mathcal{A}$ queries an index $i \in [N]$. If $i \notin \mathcal{C}$, $\mathcal{C} = \mathcal{C} \cup \{i\}$ add i to $\mathcal{C}$ and returns sk_i to $\mathcal{A}$.
   - Key substitution. The adversary $\mathcal{A}$ modifies a verification key $\mathbf{vk}[i] \leftarrow \mathsf{vk}_i^*$. If $i \notin \mathcal{C}$, $\mathcal{C} = \mathcal{C} \cup \{i\}$.
   - Signing oracle queries. The adversary $\mathcal{A}$ queries a user index $i \in [N]$, and a message $m \in \mathcal{M}$. If $i \notin \mathcal{C}$, Computes a set of $\sigma \leftarrow \mathsf{Sig}(\mathbf{vk}, \mathbf{w}, \mathsf{sk}_i, j, m)$ for every lottery index $j \in [n]$ and returns the set to $\mathcal{A}$. Records (i, m) in the set $\mathcal{Q}$.

3. The adversary $\mathcal{A}$ outputs $(\mathbf{vk}^*, m^*, \tau^*)$, where $\mathbf{vk}^* = (\mathsf{vk}_1^*, \ldots, \mathsf{vk}_N^*)$ is a vector of verification keys, $m^* \in \mathcal{M}$ is a message and τ^* is an aggregate signature. Let $\mathcal{C}^* := \{i \mid i \in \mathcal{C} \ or \ \exists \ i \ s.t. \ (i, m^*) \in \mathcal{Q}\}$.

   The adversary $\mathcal{A}$ wins if $\mathsf{ACheck}(\mathsf{avk}^*, \mathbf{vk}^*, \mathbf{w}) = 1, \mathsf{AVer}(\mathsf{avk}^*, m^*, \tau^*, t) = 1$, and $\sum_{i \in C^*} w_i < \alpha \sum_{i \in [N]} w_i$.

We say that Π_{STMS} is α-UF-CMA-s secure if for every PPT adversary $\mathcal{A}$:

$$\mathsf{Adv}_{\mathcal{A},\mathsf{STMS}}^{\alpha\textbf{-UF-CMA-s}}(\lambda) := \Pr[\alpha\textbf{-UF-CMA-s}(\lambda) = 1] < \mathsf{negl}(\lambda).$$

2.2 Instantiation

2.2.1 Mithril

We write the Mithril scheme as

$$\Pi_{\mathsf{Mithril}} := (\mathsf{Setup}, \mathsf{KGen}, \mathsf{AKey}, \mathsf{ACheck}, \mathsf{Sig}, \mathsf{ASig}, \mathsf{AVer}).$$

- $(\mathsf{pp}) \leftarrow \mathsf{Setup}(1^\lambda)$:
  run $\Pi_{\mathsf{BLS}}.\mathsf{Setup}(1^\lambda)$ to obtain $\mathsf{pp}_{\mathsf{BLS}} := (q, G_1, G_2, G_T, g_1, g_2, e, H)$

  fix a dense mapping function based on Elligator squared

$$\mathsf{Eval} : G_1 \times \mathbb{N} \times \mathcal{M} \mapsto \{0,1\}^\lambda,$$

a weighting function

$$\varphi : [0,1] \to [0,1],$$

e.g. $\varphi(w) = 1 - (1-f)^w$. Here w denotes the normalized stake fraction, and $f \in (0,1)$ is the overall winning rate.

Index range

$$n \in \mathbb{N}, \text{with } n > \log^2 \lambda.$$

A quorum or lottery parameter

$$t = n \cdot \varphi(1-\alpha).$$

Output

$$\mathsf{pp} := (\mathsf{pp}_{\mathsf{BLS}}, \mathsf{Eval}, \varphi, n, k).$$

- $(\mathsf{sk}, \mathsf{vk}) \leftarrow \mathsf{KGen}(\mathsf{pp})$:
  Parse $\mathsf{pp}_{\mathsf{BLS}}$ from pp, run $\Pi_{\mathsf{BLS}}.\mathsf{KGen}(\mathsf{pp}_{\mathsf{BLS}})$

  $$(\mathsf{sk}, \mathsf{vk}) \leftarrow \Pi_{\mathsf{BLS}}.\mathsf{KGen}(\mathsf{pp}_{\mathsf{BLS}})$$

  Output $(\mathsf{sk}, \mathsf{vk})$.

- $\mathsf{avk} \leftarrow \mathsf{AKey}(\mathbf{vk}, \mathbf{w})$:
  given

  $$\mathbf{vk} := (\mathsf{vk}_1, \ldots, \mathsf{vk}_N),$$

  $$\mathbf{w} := (w_1, \ldots, w_N),$$

  set

  $$\mathsf{avk} \leftarrow \Pi_{\mathsf{MT}}.\mathsf{Create}(\mathbf{vk}, \mathbf{w}).$$

- $b \leftarrow \mathsf{ACheck}(\mathsf{avk}, \mathbf{vk}, \mathbf{w})$, where $b \in \{0,1\}$:
  compute

  $$\mathsf{avk}' \leftarrow \Pi_{\mathsf{MT}}.\mathsf{Create}(\mathbf{vk}, \mathbf{w}).$$

  output b=1 iff $\mathsf{avk} = \mathsf{avk}'$.

- $\sigma' \leftarrow \mathsf{Sig}(\mathsf{pp}, \mathsf{avk}, \mathsf{vk}, w, \mathsf{p}, \mathsf{sk}, j, m)$:
  compute the BLS signature for eligibility check

  $$\sigma_{\mathsf{topic}} \leftarrow \Pi_{\mathsf{BLS}}.\mathsf{Sign}(\mathsf{pp}, \mathsf{sk}, m.\mathsf{topic}),$$

  compute the eligibility value

  $$ev \leftarrow \mathsf{Eval}(\sigma_{\mathsf{topic}}, j, m.\mathsf{topic}),$$

  check the validity of vk

  $$b_1 = \Pi_{\mathsf{MT}}.\mathsf{Check}(\mathsf{avk}, (\mathsf{vk}, w), \mathsf{p}).$$

If $b_1 = 1$, check the threshold

$$ev \ < \ \lfloor \varphi(w) \cdot 2^\lambda \rfloor.$$

If true, compute the BLS signature

$$\sigma \leftarrow \Pi_{\mathsf{BLS}}.\mathsf{Sign}(\mathsf{pp}, \mathsf{sk}, m),$$

output $\sigma' := (\sigma_{\mathsf{topic}}, \sigma, j)$. Otherwise output $\perp$.

- $b \leftarrow \mathsf{Ver}(\mathsf{pp}, \mathsf{avk}, \mathsf{vk}, w, \mathsf{p}, j, m, \sigma')$:
  check the validity of vk

$$b_1 = \Pi_{\mathsf{MT}}.\mathsf{Check}(\mathsf{avk}, (\mathsf{vk}, w), \mathsf{p}).$$

  parse $(\sigma_{\mathsf{topic}}, \sigma, j)$ from σ' Verify BLS signature

$$b_2 = \Pi_{BLS}.\mathsf{Ver}(\mathsf{pp}_{\mathsf{BLS}}, \mathsf{vk}, m_{\mathsf{topic}}, \sigma_{\mathsf{topic}}).$$

$$b_3 = \Pi_{BLS}.\mathsf{Ver}(\mathsf{pp}_{\mathsf{BLS}}, \mathsf{vk}, m, \sigma).$$

  Compute the eligibility value

$$ev \leftarrow \mathsf{Eval}(\sigma_{\mathsf{topic}}, j, m_{\mathsf{topic}}),$$

  and check the threshold

$$ev \ < \ \lfloor \varphi(w) \cdot 2^\lambda \rfloor.$$

  If true and $b_1 = 1, b_2 = 1, b_3 = 1$, output 1. Otherwise output 0.

- $\tau \leftarrow \mathsf{ASig}(\mathsf{pp}, \mathsf{avk}, m, \{\mathsf{vk}_{I_\eta}, \sigma'_{I_\eta}, w_{I_\eta}, p_{I_\eta}\}_{\eta=1}^t)$:
  Parse (σ, j) from each σ'. For each I_η, $\eta \in [1, t]$, check $\mathsf{Ver}(\mathsf{pp}, \mathsf{avk}, \mathsf{vk}_{I_\eta}, w_{I_\eta}, \mathsf{p}_{I_\eta}, j, m, \sigma)$.
  If any returns 0, output $\perp$.

  Set

$$\sigma^* \ = \ \prod_{\eta=1}^{t} \sigma_{I_\eta} \ \in \ G_1,$$

$$vk^* \ = \ \prod_{\eta=1}^{t} \mathsf{vk}_{I_\eta} \ \in \ G_2.$$

  Compute a seed

$$e_\sigma \ \leftarrow \ H\big(\sigma_1 \parallel \cdots \parallel \sigma_t\big),$$

  and coefficients for $\eta = 1, \ldots, t$

$$\rho_\eta \ \leftarrow \ \mathsf{H}_\lambda(e_\sigma \parallel I_\eta) \in \mathbb{Z}_q^*.$$

Compute the aggregates with seed

$$\mu \;=\; \prod_{\eta=1}^{t} \sigma_{I_\eta}^{\rho_\eta} \;\in\; G_1,$$

$$ivk \;=\; \prod_{\eta=1}^{t} vk_{I_\eta}^{\rho_\eta} \;\in\; G_2.$$

Output

$$\tau \;=\; (\,\sigma^*,\, vk^*,\, \mu,\, ivk,\, e_\sigma, \{(\mathsf{vk}_{I_\eta}, w_{I_\eta}, p_{I_\eta}, \sigma_{I_\eta})\}_{\eta=1}^{t}).$$

- $b \leftarrow \mathsf{AVer}(\mathsf{pp}, \mathsf{avk}, m, \tau, t)$, where $b \in \{0,1\}$:
  parse
  $$\tau = (\sigma^*, vk^*, \mu, ivk, e_\sigma, \{(\mathsf{vk}_{I_\eta}, w_{I_\eta}, p_{I_\eta}, \sigma_{I_\eta})\}_{\eta=1}^{t}).$$
  Check that the indices are pairwise distinct. Check that
  $$e_\sigma = H\big(\sigma_{I_1} \,\|\, \cdots \,\|\, \sigma_{I_t}\big).$$
  Compute
  $$h \leftarrow H(m).$$
  Verify the equation
  $$e(\sigma^*, g_2) \;=\; e(h, vk^*).$$
  Recompute the batching coefficients for $\eta = 1, \ldots, t$
  $$\rho_\eta \;\leftarrow\; \mathsf{H}_\lambda(e_\sigma \,\|\, I_\eta) \in \mathbb{Z}_q^*.$$
  and the corresponding aggregated public key
  $$ivk' \;=\; \prod_{\eta=1}^{t} vk_{I_\eta}^{\rho_\eta} \;\in\; G_2.$$
  Check consistency of the submitted aggregated key
  $$ivk' \;=\; ivk.$$
  Verify the batched equation
  $$e(\mu, g_2) \;=\; e(h, ivk).$$
  Output
  $$b \;=\; 1 \text{ iff all checks above hold.}$$

2.2.2 Security Proof

Theorem 2.1. *Let $n > \log^2 \lambda$, where $n \in \mathbb{N}$ be the number of lotteries. Let the quorum threshold be $t := n \cdot \varphi(1-\alpha)$, where $\varphi(w) = 1-(1-f)^w$. If the co-CDH assumption holds, the Mithril scheme described in Sec. 2.2.1 is α-UF-CMA secure in the random oracle model for $0 \le \alpha < \frac{1}{2}$.*

Proof. Let $\mathcal{A}$ be a PPT adversary with non-negligible advantage in the α-UF-CMA game. We construct a PPT algorithm $\mathcal{S}(\mathcal{A})$ that solves co-CDH problem. The co-CDH challenger samples (g_1, g_1^a, g_2^b) and gives it to $\mathcal{S}$, and $\mathcal{S}$ must output g_1^{ab}.

$\mathcal{S}$ first runs Setup of the scheme. It then fixes an upper bound N on the number of verification keys and chooses a random index $j^* \in \{1, \ldots, N\}$. For each $i \neq j^*$, $\mathcal{S}$ runs KeyGen and stores each $(\mathsf{sk}_i, \mathsf{vk}_i)$. For the special index j^*, $\mathcal{S}$ sets $\mathsf{vk}_{j^*} := g_2^b$ and stores$(\perp, \mathsf{vk}_i)$. It then assigns a stake vector $w = (w_1, \ldots, w_N)$ consistent with the model, with total stake $W = \sum_i w_i$. The public parameters, verification keys and stakes are given to $\mathcal{A}$.

When $\mathcal{A}$ issues a corruption query on some vk_i with $i \neq j^*$, $\mathcal{S}$ returns sk_i. If $\mathcal{A}$ corrupts vk_{j^*}, then $\mathcal{S}$ aborts.

Random-oracle. We assume that $\mathcal{A}$ makes at most q distinct queries and $\mathcal{S}$ guesses a random index $q^* \in \{1, \ldots, q\}$. On the q^*-th new query on some message m^*, $\mathcal{S}$ programs the oracle so that $H(m^*) = g_1^a$, hoping the message m^* will appear in the final forgery. If the final forgery of $\mathcal{A}$ is not on m^*, $\mathcal{S}$ aborts.

$\mathcal{S}$ maintains a table $\mathbf{T}$ initially empty. For each hash-to-curve query on a message m, $\mathcal{S}$ consults the table $\mathbf{T}$. If $m \in \mathbf{T}$, then $\mathcal{S}$ returns the stored value. Otherwise, $\mathcal{S}$ increments a counter c and defines $H(m)$ by setting $H(m) = g_1^a$ if $c = q^*$. Otherwise, sample a uniformaly random $s \in \mathbb{Z}_q$, set $H(m) \leftarrow g_1^s$. It then stores the pair $(m, H(m), s)$ in $\mathbf{T}$ and returns $H(m)$.

Signing-oracle. When $\mathcal{A}$ issues a signing query on input (i, m), $\mathcal{S}$ proceeds as follows. If $i = j^*$ and $m = m^*$, then $\mathcal{S}$ aborts. If $i = j^*$ and $m \neq m^*$, return $\sigma = (g_2^b)^s$ .Otherwise, $\mathcal{S}$ first ensures that the random oracle value $H(m)$ is defined by answering the hash-to-curve oracle on input m if needed, and then returns the standard BLS signature $\sigma_i := H(m)^{\mathsf{sk}_i}$.

Eventually $\mathcal{A}$ outputs a forgery consisting of a message m^*, a set of signers and stakes, and an aggregate signature σ^* that verifies correctly under the aggregate verification key and satisfies that the total underlying stake is strictly larger than αW, while the total stake of corrupted or oracle-used keys is smaller than αW.

By Lemma 1, the probability that $\mathcal{A}$ can reach the quorum threshold using only corrupted keys and oracle outputs without forging any new signature share is negligible, so any successful adversary must forge at least one signature share on m^* except with negligible probability.

Let $S := \{I_\eta\}_{\eta=1}^t$ denote the set of indices of signers underlying the forgery. The aggregation in the scheme is multiplicative in G_1, so $\sigma^* = \prod_{i \in S} H(m^*)^{\mathsf{sk}_i} = H(m^*)^{\sum_{i \in S} \mathsf{sk}_i}$. Using the known sk_i for all $i \in S \setminus \{j^*\}$, $\mathcal{S}$ computes $\sigma_{\text{others}} :=$

$\prod_{i \in S \setminus \{j^*\}} H(m^*)^{\mathsf{sk}_i}$ and then derives the individual signature of the special key as $\sigma_{j^*} := \sigma^* \cdot (\sigma_{\text{others}})^{-1} = H(m^*)^{\mathsf{sk}_{j^*}}$. By the programming of the random oracle, $H(m^*) = g_1^a$, and by the embedding, $\mathsf{vk}_{j^*} = g_2^b$, so $\sigma_{j^*} = (g_1^a)^b = g_1^{ab}$. $\mathcal{S}$ outputs σ_{j^*} as its answer to the co-CDH challenge.

Simulating the hash-to-curve queries $H(m)$ made by $\mathcal{A}$ is polynomial. The numbers of corruption and signing queries are also bounded by a polynomial in λ. Finally, extracting σ_{j^*} from σ^* and computing the corresponding product and inverse in G_1 take polynomial time as well. Hence the running time of $\mathcal{S}$ is polynomial in λ.

If $\mathcal{A}$ wins the α-$\mathsf{UF\text{-}CMA}$ game with non-negligible probability, then $\mathcal{S}$ solves co-CDH with non-negligible probability, contradicting the co-CDH assumption. Therefore, the scheme is α-$\mathsf{UF\text{-}CMA}$ secure. □

Lemma 1. Let $\alpha \in [0, \frac{1}{2})$ be the adversarial stake fraction and let $n > \log^2 \lambda$, where $n \in \mathbb{N}$ be the number of lotteries. Let the quorum threshold be $t := n \cdot \varphi(1-\alpha)$, where $\varphi(w) = 1-(1-f)^w$. Let $W_{\mathcal{A}}$ denotes the number of lotteries won by the adversary, we have

$$\Pr\left[W_{\mathcal{A}} \geq t\right] \leq \mathsf{negl}(\lambda) \ .$$

Proof. Define variable $X_\ell \in \{0,1\}$ which indicates the event that the adversary wins lottery ℓ for each $\ell \in [n]$. By assumption, the random variables $\{X_\ell\}_{\ell=1}^n$ are independent and satisfy $\Pr[X_\ell = 1] \leq \varphi(\alpha)$. Thus, the total number of wins $W_{\mathcal{A}} = \sum_{\ell=1}^n X_\ell$ can be bounded by a binomial distribution $\mathrm{Bin}(n,p)$ where $p := \varphi(\alpha)$.

Let $\mu := \mathrm{E}[W_{\mathcal{A}}] = np = n\varphi(\alpha)$. Since $\alpha < \frac{1}{2}$ and φ is increasing, we have $\varphi(1-\alpha) > \varphi(\alpha)$, and thus $t > \mu$. Define $\delta := \frac{t}{\mu} - 1 = \frac{\varphi(1-\alpha)}{\varphi(\alpha)} - 1 > 0$. Applying the multiplicative Chernoff bound, we obtain

$$\Pr\left[W_{\mathcal{A}} \geq (1+\delta)\mu\right] \leq \exp\left(-\frac{\mu\delta^2}{2+\delta}\right).$$

Substituting $\mu = n\varphi(\alpha)$ gives

$$\Pr\left[W_{\mathcal{A}} \geq t\right] \leq \exp\left(-n \cdot \frac{\varphi(\alpha)\delta^2}{2+\delta}\right).$$

Let $c := \frac{\varphi(\alpha)\delta^2}{2+\delta}$, so $c > 0$ and does not depend on λ. Then $\Pr[W_{\mathcal{A}} \geq t] \leq \exp(-cn)$. For $n > \log^2 \lambda$, we get

$$\Pr\left[W_{\mathcal{A}} \geq t\right] \leq \exp(-c\log^2 \lambda) = \lambda^{-c\log\lambda}.$$

Therefore $\Pr[W_{\mathcal{A}} \geq t]$ is negligible in λ. □

2.2.3 SNARK-based Mithril

We write the SNARK-based Mithril scheme as

$$\Pi_{\mathsf{SNARK\text{-}Mithril}} := (\mathsf{Setup}, \mathsf{KGen}, \mathsf{AKey}, \mathsf{ACheck}, \mathsf{Sig}, \mathsf{ASig}, \mathsf{AVer}).$$

It follows the same message structure $m = (m.\mathsf{topic}, m.\mathsf{body})$ as in Sec. 2.2.1.

- $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$:
  run $\Pi_{\mathsf{BLS}}.\mathsf{Setup}(1^\lambda)$ to obtain $\mathsf{pp}_{\mathsf{BLS}} := (q, G_1, G_2, G_T, g_1, g_2, e, H)$, and fix the functions Eval and φ and the lottery parameter n as in Sec. 2.2.1. Define the aggregation relation $\mathcal{R}_{\mathsf{agg}}$ (see eq. 1, below), and run
  $$\mathsf{pp}_{\mathsf{SNARK}} \leftarrow \Pi_{\mathsf{SNARK}}.\mathsf{Setup}(1^\lambda, \mathcal{R}_{\mathsf{agg}}).$$
  Output
  $$\mathsf{pp} := (\mathsf{pp}_{\mathsf{BLS}}, \mathsf{pp}_{\mathsf{SNARK}}, \mathsf{Eval}, \varphi, \mathsf{H}_\lambda, \mathsf{H}_{\mathsf{seed}}, n, t).$$
- $(\mathsf{sk}, \mathsf{vk}) \leftarrow \mathsf{KGen}(\mathsf{pp})$:
  Parse $\mathsf{pp}_{\mathsf{BLS}}$ from pp, run $\Pi_{\mathsf{BLS}}.\mathsf{KGen}(\mathsf{pp}_{\mathsf{BLS}})$
  $$(\mathsf{sk}, \mathsf{vk}) \leftarrow \Pi_{\mathsf{BLS}}.\mathsf{KGen}(\mathsf{pp}_{\mathsf{BLS}})$$
  Output $(\mathsf{sk}, \mathsf{vk})$.
- $\mathsf{avk} \leftarrow \mathsf{AKey}(\mathbf{vk}, \mathbf{w})$:
  given
  $$\mathbf{vk} := (\mathsf{vk}_1, \ldots, \mathsf{vk}_N),$$
  $$\mathbf{w} := (w_1, \ldots, w_N),$$
  set
  $$\mathsf{avk} \leftarrow \Pi_{\mathsf{MT}}.\mathsf{Create}(\mathbf{vk}, \mathbf{w}).$$
- $b \leftarrow \mathsf{ACheck}(\mathsf{avk}, \mathbf{vk}, \mathbf{w})$, where $b \in \{0, 1\}$:
  compute
  $$\mathsf{avk}' \leftarrow \Pi_{\mathsf{MT}}.\mathsf{Create}(\mathbf{vk}, \mathbf{w}).$$
  output b=1 iff $\mathsf{avk} = \mathsf{avk}'$.
- $\sigma' \leftarrow \mathsf{Sig}(\mathsf{pp}, \mathsf{avk}, \mathsf{vk}, w, \mathsf{p}, \mathsf{sk}, j, m)$:
  compute the BLS signature for eligibility check
  $$\sigma_{\mathsf{topic}} \leftarrow \Pi_{\mathsf{BLS}}.\mathsf{Sign}(\mathsf{pp}, \mathsf{sk}, m.\mathsf{topic}),$$
  compute the eligibility value
  $$ev \leftarrow \mathsf{Eval}(\sigma_{\mathsf{topic}}, j, m.\mathsf{topic}),$$
  check the validity of vk
  $$b_1 = \Pi_{\mathsf{MT}}.\mathsf{Check}(\mathsf{avk}, (\mathsf{vk}, w), \mathsf{p}).$$
  If $b_1 = 1$, check the threshold
  $$ev \ < \ \lfloor \varphi(w) \cdot 2^\lambda \rfloor.$$
  If true, compute the BLS signature
  $$\sigma \leftarrow \Pi_{\mathsf{BLS}}.\mathsf{Sign}(\mathsf{pp}, \mathsf{sk}, m),$$
  output $\sigma' := (\sigma_{\mathsf{topic}}, \sigma, j)$. Otherwise output $\perp$.

- $b \leftarrow \mathsf{Ver}(\mathsf{pp}, \mathsf{avk}, \mathsf{vk}, w, \mathsf{p}, j, m, \sigma')$:
  check the validity of vk

$$b_1 = \Pi_{\mathsf{MT}}.\mathsf{Check}(\mathsf{avk}, (\mathsf{vk}, w), \mathsf{p}).$$

  parse $(\sigma_{\mathsf{topic}}, \sigma, j)$ from σ' Verify BLS signature

$$b_2 = \Pi_{BLS}.\mathsf{Ver}(\mathsf{pp}_{\mathsf{BLS}}, \mathsf{vk}, m_{\mathsf{topic}}, \sigma_{\mathsf{topic}}).$$

$$b_3 = \Pi_{BLS}.\mathsf{Ver}(\mathsf{pp}_{\mathsf{BLS}}, \mathsf{vk}, m, \sigma).$$

  Compute the eligibility value

$$ev \leftarrow \mathsf{Eval}(\sigma_{\mathsf{topic}}, j, m_{\mathsf{topic}}),$$

  and check the threshold

$$ev \ < \ \lfloor \varphi(w) \cdot 2^{\lambda} \rfloor.$$

  If true and $b_1 = 1, b_2 = 1, b_3 = 1$, output 1. Otherwise output 0.

- $\tau \leftarrow \mathsf{ASig}(\mathsf{pp}, \mathsf{avk}, m, \{(\sigma'_{I_\eta}, p_{I_\eta})\}_{\eta=1}^{t})$:
  parse $\mathsf{pp}_{\mathsf{BLS}}, \mathsf{pp}_{\mathsf{SNARK}}, \mathsf{Eval}, \varphi, \mathsf{H}_\lambda, \mathsf{H}_{\mathsf{seed}}$ from pp. For each $\eta \in [t]$, parse $\sigma'_{I_\eta} = (\sigma_{\mathsf{topic},I_\eta}, \sigma_{I_\eta}, j_\eta)$. Compute

$$e_\sigma \leftarrow \mathsf{H}_{\mathsf{seed}}(\sigma_{I_1} \parallel \cdots \parallel \sigma_{I_t}) \in \{0,1\}^{\lambda}, \qquad \rho_\eta \leftarrow \mathsf{H}_\lambda(e_\sigma \parallel I_\eta) \in \mathbb{Z}_q^*,$$

  and set

$$\mu := \prod_{\eta=1}^{t} \sigma_{I_\eta}^{\rho_\eta} \in G_1, \qquad ivk := \prod_{\eta=1}^{t} vk_{I_\eta}^{\rho_\eta} \in G_2.$$

  Let the statement be $x := (\mathsf{avk}, m, \mu, ivk, e_\sigma)$ and the witness be

$$w := \big(\{(I_\eta, vk_{I_\eta}, w_{I_\eta}, p_{I_\eta}, \sigma_{\mathsf{topic},I_\eta}, \sigma_{I_\eta}, j_\eta)\}_{\eta=1}^{t}\big).$$

  Define $\mathcal{R}_{\mathsf{agg}}$ as

$$\begin{aligned}
\mathcal{R}_{\mathsf{agg}} = \{(x,w) \mid\ & \Pi_{\mathsf{MT}}.\mathsf{Check}(\mathsf{avk}, (vk_{I_\eta}, w_{I_\eta}), p_{I_\eta}) = 1 \\
& \wedge\ \Pi_{\mathsf{BLS}}.\mathsf{Ver}(\mathsf{pp}_{\mathsf{BLS}}, vk_{I_\eta}, m.\mathsf{topic}, \sigma_{\mathsf{topic},I_\eta}) = 1 \\
& \wedge\ \Pi_{\mathsf{BLS}}.\mathsf{Ver}(\mathsf{pp}_{\mathsf{BLS}}, vk_{I_\eta}, m, \sigma_{I_\eta}) = 1 \\
& \wedge\ \mathsf{Eval}(\sigma_{\mathsf{topic},I_\eta}, j_\eta, m.\mathsf{topic}) \ < \ \lfloor \varphi(w_{I_\eta}) \cdot 2^{\lambda} \rfloor \\
& \wedge\ \mu = \prod_{\eta=1}^{t} \sigma_{I_\eta}^{\rho_\eta} \wedge ivk = \prod_{\eta=1}^{t} vk_{I_\eta}^{\rho_\eta}\}
\end{aligned} \tag{1}$$

  Compute

$$\pi \leftarrow \Pi_{\mathsf{SNARK}}.\mathsf{Prove}(\mathsf{pp}_{\mathsf{SNARK}}, x, w),$$

  and output

$$\tau := (\mu, ivk, e_\sigma, \pi).$$

- $b \leftarrow \mathsf{AVer}(\mathsf{pp}, \mathsf{avk}, m, \tau)$, where $b \in \{0,1\}$:
  parse $\mathsf{pp}_{\mathsf{BLS}}, \mathsf{pp}_{\mathsf{SNARK}}$ from pp and parse $\tau = (\mu, ivk, e_\sigma, \pi)$. Set $h \leftarrow H(m)$ and check
  $$e(\mu, g_2) = e(h, ivk),$$
  and
  $$\Pi_{\mathsf{SNARK}}.\mathsf{Verify}(\mathsf{pp}_{\mathsf{SNARK}}, (\mathsf{avk}, m, \mu, ivk, e_\sigma), \pi) = 1.$$
  Output $b = 1$ iff both checks hold.

2.2.4 Security Proof

Theorem 2.2. *Let $\alpha \in [0, \frac{1}{2})$ and $n > \log^2 \lambda$, where $n \in \mathbb{N}$ be the number of lotteries. Let the quorum threshold be $t := n \cdot \varphi(1-\alpha)$, where $\varphi(w) = 1-(1-f)^w$. If co-CDH and extractability of SNARK holds, $\Pi_{\mathsf{SNARK\text{-}Mithril}}$ is α-UF-CMA secure in the random oracle model.*

Proof. Let $\mathcal{A}$ be a PPT adversary that wins the α-UF-CMA experiment against $\Pi_{\mathsf{SNARK\text{-}Mithril}}$ with non-negligible probability. We construct a PPT algorithm $\mathcal{S}(\mathcal{A})$ that, given a co-CDH challenge (g_1, g_1^a, g_2^b), outputs g_1^{ab} with non-negligible probability.

$\mathcal{S}$ runs Setup and uses the bilinear groups and generators from the challenge as the BLS public parameters. As in the previous security proof, $\mathcal{S}$ samples a uniform special index $j^* \xleftarrow{\$} [N]$ and embeds the co-CDH instance by setting $\mathsf{vk}_{j^*} := g_2^b$ while generating all other keys honestly: for each $i \neq j^*$, sample $\mathsf{sk}_i \xleftarrow{\$} \mathbb{Z}_q$ and set $\mathsf{vk}_i := g_2^{\mathsf{sk}_i}$.

$\mathcal{S}$ simulates the random oracle $H : \{0,1\}^* \to G_1$ via sampling with a table $\mathbf{T}_H$, and guesses a uniform index $q^* \in \{1, \ldots, q\}$, where q upper-bounds the number of distinct H-queries made by $\mathcal{A}$. It maintains a counter ctr initially 0. On a fresh query $H(m)$, it increments ctr and programs the oracle at the guessed position by setting $H(m) = g_1^a$ if $\mathsf{ctr} = q^*$, and otherwise samples $s \xleftarrow{\$} \mathbb{Z}_q$ and returns $H(m) := g_1^s$. Let $m^\dagger$ denote the unique message at which H was programmed to g_1^a, i.e., the q^*-th new query.

$\mathcal{S}$ answers H_λ and $\mathsf{H}_{\mathsf{seed}}$ as random oracles by sampling with tables, without any programming. For corruption and key-substitution queries, $\mathcal{S}$ answers honestly for all $i \neq j^*$, and aborts if $\mathcal{A}$ corrupts j^* or substitutes $\mathbf{vk}[j^*]$, since this would destroy the embedding $\mathsf{vk}_{j^*} = g_2^b$. For signing-oracle queries, $\mathcal{S}$ answers honestly for all $i \neq j^*$ by computing the required BLS signatures using sk_i. If $\mathcal{A}$ requests a signing query for $i = j^*$, $\mathcal{S}$ aborts, since it cannot sign under vk_{j^*}.

Eventually $\mathcal{A}$ outputs a forgery $(\mathsf{avk}^*, \mathbf{vk}^*, m^*, \tau^*)$, where $\tau^* = (\mu^*, ivk^*, e_\sigma^*, \pi^*)$ is accepted by AVer for $(\mathsf{pp}, \mathsf{avk}^*, m^*)$. If $m^* \neq m^\dagger$, then $\mathcal{S}$ aborts. Otherwise, by acceptance we have $\Pi_{\mathsf{SNARK}}.\mathsf{Verify}(\mathsf{pp}_{\mathsf{SNARK}}, x^*, \pi^*) = 1$ for the statement $x^* := (\mathsf{avk}^*, m^*, \mu^*, ivk^*, e_\sigma^*)$.

By SNARK extractability, $\mathcal{S}$ invokes the extractor to obtain a witness w^* such that $(x^*, w^*) \in \mathcal{R}_{\mathsf{agg}}$, where w^* contains tuples $(I_\eta, vk_{I_\eta}, w_{I_\eta}, p_{I_\eta}, \sigma_{\mathsf{topic},I_\eta}, \sigma_{I_\eta}, j_\eta)_{\eta=1}^t$ that satisfy all checks encoded by $\mathcal{R}_{\mathsf{agg}}$ and are consistent with the aggregation equations under coefficients $\rho_\eta := \mathsf{H}_\lambda(e_\sigma^* \| I_\eta)$. Let $S := \{I_\eta\}_{\eta=1}^t$ be the extracted

signer-index multiset. By the same lottery argument used in the previous proof under the threshold choice $t = n \cdot \varphi(1-\alpha)$, except with negligible probability there exists at least one extracted index in $S \setminus \mathcal{C}^*$, and since j^* is uniform in $[N]$ we have $\Pr[j^* \in S \setminus \mathcal{C}^*] \geq 1/N$. Condition on the event that $j^* \in S \setminus \mathcal{C}^*$.

Now $\mathcal{S}$ recomputes all coefficients ρ_η using its simulation of H_λ, computes $\mu_{\text{others}} := \prod_{\eta:\ I_\eta \neq j^*} \sigma_{I_\eta}^{\rho_\eta}$, isolates $\mu_{j^*} := \mu^* \cdot \mu_{\text{others}}^{-1}$, and obtains $\sigma_{j^*} := (\mu_{j^*})^{\rho_{j^*}^{-1}}$, where ρ_{j^*} is the coefficient associated with the index j^* in the extracted witness. Since $(x^*, w^*) \in \mathcal{R}_{\mathsf{agg}}$, the element σ_{j^*} is a valid BLS signature on m^* under $\mathsf{vk}_{j^*} = g_2^b$, hence $\sigma_{j^*} = H(m^*)^b$. By our programming we have $H(m^*) = H(m^\dagger) = g_1^a$, and therefore $\sigma_{j^*} = (g_1^a)^b = g_1^{ab}$, which $\mathcal{S}$ outputs as the co-CDH solution.

It remains to bound the abort probability. $\mathcal{S}$ aborts only if $\mathcal{A}$ corrupts or substitutes j^*, or requests a signing query for j^*, or if $\mathcal{S}$ guesses the wrong H-query to program (i.e., $m^* \neq m^\dagger$), or if SNARK extraction fails (negligible by assumption). Since j^* and q^* are guessed uniformly and all bounds are polynomial, $\mathcal{S}$ succeeds with probability at least $\Pr[\mathcal{A}\ \mathsf{wins}]/\mathsf{poly}(\lambda) - \mathsf{negl}(\lambda)$. Thus any non-negligible forging probability of $\mathcal{A}$ would imply a non-negligible success probability for $\mathcal{S}$ in solving co-CDH, contradicting the co-CDH assumption. Hence $\Pi_{\mathsf{SNARK\text{-}Mithril}}$ is α-$\mathsf{UF\text{-}CMA}$ secure. □

2.2.5 Schnorr's Mithril

$$\Pi_{\mathsf{Sch\text{-}Mithril}} := (\mathsf{Setup}, \mathsf{KGen}, \mathsf{AKey}, \mathsf{ACheck}, \mathsf{Sig}, \mathsf{Ver}, \mathsf{ASig}, \mathsf{AVer}).$$

$$m = (m.\mathsf{topic}, m.\mathsf{body}) \in \mathcal{M}.$$

- $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$:

$$(q, G_1, g_1) \text{ with } |G_1| = q,$$
$$H_1 : \{0,1\}^* \to G_1, \qquad H_2 : G_1^* \to \mathbb{Z}_q,$$
$$\mathsf{Eval} : G_1 \times \mathbb{N} \times \mathcal{M} \to \{0,1\}^\lambda,$$
$$\varphi : [0,1] \to [0,1],$$
$$n \in \mathbb{N}, \qquad n > \log^2 \lambda,$$
$$t = n \cdot \varphi(1-\alpha),$$
$$\mathsf{pp} := (q, G_1, g_1, H_1, H_2, \mathsf{Eval}, \varphi, n, t).$$

- $(\mathsf{sk}, \mathsf{vk}) \leftarrow \mathsf{KGen}(\mathsf{pp})$:

$$\mathsf{sk} \xleftarrow{\$} \mathbb{Z}_q, \qquad \mathsf{vk} := g_1^{\mathsf{sk}} \in G_1.$$

- $\mathsf{avk} \leftarrow \mathsf{AKey}(\mathbf{vk}, \mathbf{w})$:

$$\mathsf{avk} \leftarrow \Pi_{\mathsf{MT}}.\mathsf{Create}\big(\{(\mathsf{vk}_i, w_i)\}_{i=1}^N\big).$$

- $b \leftarrow \mathsf{ACheck}(\mathsf{avk}, \mathbf{vk}, \mathbf{w})$:

$$\mathsf{avk}' := \Pi_{\mathsf{MT}}.\mathsf{Create}\big(\{(\mathsf{vk}_i, w_i)\}_{i=1}^{N}\big), \qquad b := \mathbf{1}[\mathsf{avk} = \mathsf{avk}'].$$

- $\sigma' \leftarrow \mathsf{Sig}(\mathsf{pp}, \mathsf{avk}, \mathsf{vk}, w, \mathsf{p}, \mathsf{sk}, j, m)$:

$$b_1 := \Pi_{\mathsf{MT}}.\mathsf{Check}(\mathsf{avk}, (\mathsf{vk}, w), \mathsf{p}),$$

$$\sigma_{\mathsf{topic}} := H_1(m.\mathsf{topic})^{\mathsf{sk}}, ev := \mathsf{Eval}(\sigma_{\mathsf{topic}}, j, m.\mathsf{topic}), b_2 := (ev < \lfloor \varphi(w) \cdot 2^{\lambda} \rfloor),$$

$$\sigma := H_1(m)^{\mathsf{sk}}, r \xleftarrow{\$} \mathbb{Z}_q, R_1 := H_1(m)^r, R_2 := g_1^r,$$

$$c := H_2(g_1, H_1(m), \mathsf{vk}, \sigma, R_1, R_2), s := r + \mathsf{sk} \cdot c \ \in \mathbb{Z}_q, \pi := (s, c),$$

$$\sigma' := (\sigma_{\mathsf{topic}}, \sigma, \pi, j, R_1, R_2), \sigma' := \perp \text{ if } b_1 = 0 \text{ or } b_2 = 0.$$

- $b \leftarrow \mathsf{Ver}(\mathsf{pp}, \mathsf{avk}, \mathsf{vk}, w, \mathsf{p}, j, m, \sigma')$: parse $(\sigma_{\mathsf{topic}}, \sigma, (s, c), j, R_1, R_2)$ from σ',

$$b_1 := \Pi_{\mathsf{MT}}.\mathsf{Check}(\mathsf{avk}, (\mathsf{vk}, w), \mathsf{p}),$$

$$ev := \mathsf{Eval}(\sigma_{\mathsf{topic}}, j, m.\mathsf{topic}), \qquad b_1 := \mathbf{1}\big[ev < \lfloor \varphi(w) \cdot 2^{\lambda} \rfloor\big],$$

$$\widetilde{R}_1 := H_1(m)^s \cdot \sigma^{-c}, \qquad \widetilde{R}_2 := g_1^s \cdot \mathsf{vk}^{-c},$$

$$\widetilde{c} := H_2(g_1, H_1(m), \mathsf{vk}, \sigma, \widetilde{R}_1, \widetilde{R}_2), \qquad b_{\mathsf{msg}} := \mathbf{1}[c = \widetilde{c}],$$

$$b := \mathbf{1}[b_1 = 1 \wedge b_2 = 1 \wedge b_{\mathsf{topic}} = 1 \wedge b_{\mathsf{msg}} = 1].$$

- $\tau \leftarrow \mathsf{ASig}(\mathsf{pp}, \mathsf{avk}, m, \{(\mathsf{vk}_{I_\eta}, w_{I_\eta}, p_{I_\eta}, \sigma'_{I_\eta})\}_{\eta=1}^{t})$:

$$\rho := H_2(c_1, ..., c_t, s_1, ..., s_t),$$

$$\text{for } 1 \le \eta \le t, \quad a_{\eta,0} := H_3(\rho, \eta, 0), \quad a_{\eta,1} := H_3(\rho, \eta, 1)$$

$$\sigma^* := \prod_{\eta=1}^{t} \sigma_{I_\eta}^{c_\eta a_{\eta,0}} \in G_1, \qquad ivk := \prod_{\eta=1}^{t} vk_{I_\eta}^{c_\eta a_{\eta,1}} \in G_1,$$

$$R_1^* := \prod_{\eta=1}^{t} R_{1,I_\eta}^{c_\eta a_{\eta,0}} \in G_1, \qquad R_2^* := \prod_{\eta=1}^{t} R_{2,I_\eta}^{c_\eta a_{\eta,1}} \in G_1,$$

$$H_1(msg)^* := H_1(msg)^{\sum_{\eta=1}^{t} c_\eta a_{\eta,0}}, \qquad g_1^* := g_1^{\sum_{\eta=1}^{t} c_\eta a_{\eta,1}},$$

$$\tau := (\sigma^*, ivk, R_1^*, R_2^*, H_1(msg)^*, g_1^*).$$

- $b \leftarrow \mathsf{AVer}(\mathsf{pp}, \mathsf{avk}, m, \tau, t)$: parse $(\sigma^*, ivk, R_1^*, R_2^*, H_1(msg)^*, g_1^*)$ from τ,

$$R_1^* R_2^* \sigma^* ivk = H_1(msg)^* g_1^*.$$